

# DSA Code Notes

Data Structures & Algorithms – All 12 Programs

## 1. Library Data Analysis

• Dict madhe members ani books store kele. Average, most/least borrowed book kadhle.

```
library_data = {
    "M001": ["Book A", "Book B"], "M002": ["Book C"],
    "M003": [], "M004": ["Book A", "Book A"],
    "M005": [], "M006": ["Book B", "Book D"]
}
total_member = len(library_data)
total_books = sum(len(b) for b in library_data.values())
avg = total_books / total_member
print("Average:", avg)

book_counts = {}
for books in library_data.values():
    for book in books:
        book_counts[book] = book_counts.get(book, 0) + 1

most = max(book_counts, key=book_counts.get)
least = min(book_counts, key=book_counts.get)
print("Most borrowed:", most)
print("Least borrowed:", least)
```

## 2. Linear & Binary Search

• Customer ID list madhe linear ani binary search kela. Binary search saathi list sort karna padto.

```
customer_id = [100, 102, 101]

def linear_search(lst, target):
    for i in range(len(lst)):
        if lst[i] == target:
            return True
    return False

def binary_search(lst, target):
    low, high = 0, len(lst) - 1
    while low <= high:
        mid = (low + high) // 2
        if lst[mid] == target: return True
        elif lst[mid] < target: low = mid + 1
        else: high = mid - 1
    return False

target = int(input("Enter customer ID: "))
print("Linear:", "Found" if linear_search(customer_id, target) else "Not Found")
sorted_ids = sorted(customer_id)
print("Binary:", "Found" if binary_search(sorted_ids, target) else "Not Found")
```

## 3. Selection & Bubble Sort

- Salary list selection sort ani bubble sort ne sort keli. Top 5 descending order madhe print kele.

```
salaries = [10000, 12000, 14000, 9000, 14000, 98712, 461452, 4826]
```

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j # Bug fix: was min_index + j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

print(selection_sort(salaries.copy())[-5][::-1])
print(bubble_sort(salaries.copy())[-5][::-1])
```

## 4. Text Editor – Undo/Redo System

- Stack use kela undo/redo saathi. Make change → undo stack la push. Undo → redo stack la transfer.

```
class TextEditor:
    def __init__(self):
        self.document = ""
        self.undo_stack = []
        self.redo_stack = []

    def make_change(self, change):
        self.undo_stack.append(self.document)
        self.document += change
        self.redo_stack.clear()

    def undo(self):
        if self.undo_stack:
            self.redo_stack.append(self.document)
            self.document = self.undo_stack.pop()

    def redo(self):
        if self.redo_stack:
            self.undo_stack.append(self.document)
            self.document = self.redo_stack.pop()

    def show(self):
        print("Document:", self.document)

editor = TextEditor()
editor.make_change("Hello ")
editor.make_change("World")
```

```
editor.show() # Hello World
editor.undo()
editor.show() # Hello
editor.redo()
editor.show() # Hello World
```

## 5. Real-Time Event Processing (Queue)

- List la queue sarkha vaparla. FIFO order – pop(0) ne first event process hoto.

```
event_queue = []

def add_event(event):
    event_queue.append(event)
    print("Added:", event)

def process_next():
    if event_queue:
        print("Processed:", event_queue.pop(0))
    else:
        print("No events.")

def show_pending():
    print("Pending:", event_queue if event_queue else "None")

def cancel_event(name):
    if name in event_queue:
        event_queue.remove(name)
        print("Cancelled:", name)
    else:
        print("Not found:", name)

add_event("Login")
add_event("Payment")
add_event("Logout")
show_pending()
process_next()
cancel_event("Logout")
show_pending()
```

## 6. Student Record Management (Linked List)

- Singly linked list vaparli. Each node madhe roll\_no, name, marks ani next pointer asto.

```
class Node:
    def __init__(self, roll, name, marks):
        self.roll = roll; self.name = name
        self.marks = marks; self.next = None

class StudentList:
    def __init__(self): self.head = None

    def add(self, roll, name, marks):
        node = Node(roll, name, marks)
        if not self.head: self.head = node
```

```

else:
    cur = self.head
    while cur.next: cur = cur.next
    cur.next = node

def delete(self, roll):
    cur, prev = self.head, None
    while cur:
        if cur.roll == roll:
            if prev: prev.next = cur.next
            else: self.head = cur.next
            return "Deleted"
        prev, cur = cur, cur.next
    return "Not found"

def search(self, roll):
    cur = self.head
    while cur:
        if cur.roll == roll:
            print(f"{cur.roll} | {cur.name} | {cur.marks}")
            return
        cur = cur.next
    print("Not found")

s = StudentList()
s.add(1, "Rahul", 85)
s.add(2, "Priya", 92)
s.search(1)
s.delete(1)
s.search(1)

```

## 7. Hash Table – Division Method (Chaining)

•  $key \% size$  ne index kadhto. Collision saathi chaining – each slot la list asto.

```

class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key): return key % self.size

    def insert(self, key, value):
        idx = self._hash(key)
        for pair in self.table[idx]:
            if pair[0] == key:
                pair[1] = value; return
        self.table[idx].append([key, value])

    def search(self, key):
        for pair in self.table[self._hash(key)]:
            if pair[0] == key: return pair[1]
        return None

    def delete(self, key):

```

```

idx = self._hash(key)
self.table[idx] = [p for p in self.table[idx] if p[0] != key]

ht = HashTable()
ht.insert(15, "apple")
ht.insert(25, "banana") # Collision with 15
print(ht.search(25)) # banana
ht.delete(25)
print(ht.search(25)) # None

```

## 8. Linear Probing Hash Table

• Open addressing – collision zhali tar next slot bagha. DELETED marker vaparla safe delete saathi.

```

class LinearProbingHT:
    def __init__(self, size=10):
        self.size = size
        self.table = [None] * size
        self.DEL = "<DELETED>"

    def _hash(self, key): return key % self.size

    def insert(self, key):
        idx = self._hash(key)
        start = idx
        while self.table[idx] not in (None, self.DEL):
            if self.table[idx] == key: return # Already exists
            idx = (idx + 1) % self.size
        if idx == start: return # Table full
        self.table[idx] = key

    def search(self, key):
        idx = self._hash(key)
        start = idx
        while self.table[idx] is not None:
            if self.table[idx] == key: return idx
            idx = (idx + 1) % self.size
        if idx == start: break
        return None

    def delete(self, key):
        idx = self.search(key)
        if idx is not None: self.table[idx] = self.DEL

ht = LinearProbingHT()
ht.insert(10); ht.insert(20); ht.insert(30)
print(ht.search(20)) # index number
ht.delete(20)
print(ht.search(20)) # None

```

## 9. Extendible Hashing (Bucket)

• Dynamic hashing – bucket full zhala ki split hoto. Global depth vadhto when needed.

```

class Bucket:
def __init__(self, depth, size):
self.depth = depth; self.size = size; self.items = {}
def is_full(self): return len(self.items) >= self.size
def insert(self, k, v): self.items[k] = v
def search(self, k): return self.items.get(k)
def delete(self, k): self.items.pop(k, None)

class ExtHashTable:
def __init__(self, bsize=2):
self.global_depth = 1; self.bsize = bsize
self.dir = [Bucket(1, bsize) for _ in range(2)]

def _hash(self, k): return hash(k) & ((1 << self.global_depth) - 1)

def insert(self, k, v):
idx = self._hash(k); b = self.dir[idx]
if k in b.items or not b.is_full():
b.insert(k, v)
else:
self._split(idx); self.insert(k, v)

def _split(self, idx):
old = self.dir[idx]; ld = old.depth; old.depth += 1
if old.depth > self.global_depth:
self.dir += self.dir; self.global_depth += 1
new = Bucket(old.depth, self.bsize)
for i in range(len(self.dir)):
if self.dir[i] is old and ((i >> ld) & 1):
self.dir[i] = new
items = list(old.items.items()); old.items.clear()
for k, v in items: self.insert(k, v)

def search(self, k): return self.dir[self._hash(k)].search(k)
def delete(self, k): self.dir[self._hash(k)].delete(k)

ht = ExtHashTable(2)
for i in range(1, 6): ht.insert(i, f"val{i}")
print(ht.search(3)) # val3
ht.delete(3)
print(ht.search(3)) # None

```

## 10. DFS & BFS Graph Traversal

• DFS – adjacency matrix use kela (recursive). BFS – adjacency list ani deque use keli.

```

from collections import deque

locations = ['A', 'B', 'C', 'D']
loc_idx = {n: i for i, n in enumerate(locations)}

adj_matrix = [
[0, 1, 1, 0], # A
[1, 0, 0, 1], # B
[1, 0, 0, 1], # C
[0, 1, 1, 0] # D

```

```

]

def dfs(start):
    visited = [False] * 4; result = []
    def _dfs(node):
        visited[node] = True; result.append(locations[node])
        for nb in range(4):
            if adj_matrix[node][nb] == 1 and not visited[nb]:
                _dfs(nb)
    _dfs(loc_idx[start]); return result

adj_list = {'A': ['B','C'], 'B': ['A','D'], 'C': ['A','D'], 'D': ['B','C']}

def bfs(start):
    visited = set(); q = deque([start]); result = []
    while q:
        node = q.popleft()
        if node not in visited:
            visited.add(node); result.append(node)
            q.extend(nb for nb in adj_list[node] if nb not in visited)
    return result

print("DFS:", dfs('A'))
print("BFS:", bfs('A'))

```

## 11. Binary Search Tree (Basic)

• *BST – left < root < right. Insert, search, inorder traversal ani delete (3 cases) implement kele.*

```

class Node:
    def __init__(self, key):
        self.key = key; self.left = self.right = None

class BST:
    def __init__(self): self.root = None

    def insert(self, root, key):
        if not root: return Node(key)
        if key < root.key: root.left = self.insert(root.left, key)
        elif key > root.key: root.right = self.insert(root.right, key)
        return root

    def search(self, root, key):
        if not root or root.key == key: return root
        return self.search(root.left if key < root.key else root.right, key)

    def inorder(self, root):
        if root:
            self.inorder(root.left); print(root.key, end=" "); self.inorder(root.right)

    def delete(self, root, key):
        if not root: return root
        if key < root.key: root.left = self.delete(root.left, key)
        elif key > root.key: root.right = self.delete(root.right, key)
        else:

```

```

if not root.left: return root.right
if not root.right: return root.left
mn = root.right
while mn.left: mn = mn.left
root.key = mn.key
root.right = self.delete(root.right, mn.key)
return root

```

```

bst = BST(); root = None
for v in [50, 30, 70, 20, 40, 60, 80]:
    root = bst.insert(root, v)
bst.inorder(root) # 20 30 40 50 60 70 80
root = bst.delete(root, 30)
bst.inorder(root) # 20 40 50 60 70 80

```

## 12. City Population BST

- City name ne BST banvla. Alphabetical order madhe insert, search (comparison count), update, delete.

```

class Node:
def __init__(self, city, pop):
self.city = city; self.pop = pop; self.left = self.right = None

class CityBST:
def __init__(self): self.root = None

def insert(self, root, city, pop):
if not root: return Node(city, pop)
c = city.lower()
if c < root.city.lower(): root.left = self.insert(root.left, city, pop)
elif c > root.city.lower(): root.right = self.insert(root.right, city, pop)
else: root.pop = pop # Update if exists
return root

def search(self, root, city):
cur = root; comp = 0
while cur:
comp += 1
if city.lower() == cur.city.lower(): return cur, comp
cur = cur.left if city.lower() < cur.city.lower() else cur.right
return None, comp

def inorder(self, root):
if root:
self.inorder(root.left)
print(f" {root.city}: {root.pop}")
self.inorder(root.right)

bst = CityBST(); root = None
for city, pop in [("Mumbai", 20000000), ("Pune", 6000000), ("Nagpur", 2500000), ("Nashik", 1500000)]:
    root = bst.insert(root, city, pop)

print("Cities A-Z:"); bst.inorder(root)
node, comp = bst.search(root, "Pune")
print(f"Found: {node.city} ({node.pop}), Comparisons: {comp}")

```



